

Cicci Research

Arkeo All-in-One API

v0.1.0

03/10/2025

Contents

Introduction	2
List of commands	3
MAIN	4
Command Summary	4
Commands Details	4
ROUTINE.....	6
Command Summary	6
Commands Details	6
Routine Specific Commands	10
TMPCTL	11
MUX	12
Command Summary	12
Commands Details	12
Examples	15
Python	15
Changelog.....	16

Introduction

This API facilitates remote control of the Arkeo All-in-One software. Communication happens using the TCP protocol on port 6360, enabling users to perform a variety of commands manage measurements. Messages are formatted as JSON, its structure depends on the specific commands.

Each command must sent as a JSON string in the form

```
{  
  "target":<TARGET>,  
  "command": "<COMMAND>",  
  "parameter": "<PARAMETERS>",  
  "request_id": 42          // optional  
}
```

Each command must be preceded by a 4-byte integer indicating the total length of the command. After each command, a reply is always sent back with the following structure:

```
{  
  "status": "OK" | "ERROR",  
  "data": { ... },          // present if status = "OK"  
  "error": { ... },         // present if status = "ERROR"  
  "request_id": <same as request>  
}
```

The "data" format depends on the command. If an unknown command is sent or the JSON is malformed, "status" will indicate "error" instead and the error key is send instead of the data key.

List of commands

Each "target" has its own set of commands. The routine target may have custom commands based on which routine is active. Sending a command not in the list of available commands for the selected target, returns "invalid command".

The following sections details every available command, separated by targets. The following targets are available

<u>MAIN</u>	Commands related to the main launcher
<u>ROUTINE</u>	Commands for the active routine
<u>TBA</u>	Commands for a temperature controller
<u>TMPCTL</u>	
<u>MUX</u>	Commands for a multiplexer

Each command detail includes an example request and response. "target", "status" and "request_id" are not included in the examples, but are implied.

MAIN

These commands can always be called.

Command Summary

Command	Description
StartRoutine	Starts the selected routine
GetActiveRoutine	Returns the active routine. Only one routine may be active at a time
GetAvailableRoutines	Returns an array of all available routines

Commands Details

Command: StartRoutine

Description: Launches a routine.

Parameters

Key	Type	Required	Description
routine_name	string	✓	Name of the routine to load.

Response

Key	Type	Description
state	string	OK if routine started Error if a routine is already active

Example Request

```
{
  "command": "StartRoutine",
  "params": { "routine_name": "JV" },
}
```

Example Response

```
{ "state": "OK" }
```

Command: GetActiveRoutine

Description: Returns the active routine. Only one routine may be active at a time.

Response

Key	Type	Description
state	string	Name of the active routine, or empty if none.

Example Request

```
{ "command": "GetActiveRoutine" }
```

Example Response

```
{"routine_name": "JV" }
```

Command: GetAvailableRoutines

Description: Returns an array of all available routines.

Response

Key	Type	Description
routines	string[]	Array of available routine names.

Example Request

```
{ "command": "GetAvailableRoutines" }
```

Example Response

```
{ "routines":["JV","MPPT"] }
```

ROUTINE

These commands can only be called if a routine is active. If no routine is active, “no active routine” is returned.

Command Summary

Command	Description
StartMeasurement	Starts the measurements
StopMeasurement	Stops the ongoing measurement
CloseRoutine	Stops any ongoing measurement and close the routine
GetTestData	Returns the measurement data
GetTestStatus	Returns the current status of the routine
ClearErrors	Clears any errors when the routine is in the error state
GetSettings	Returns the routine configuration
ApplySettings	Applies a configuration
SetInfo	Applies user and device name

Commands Details

Command: StartMeasurement

Description: Starts the measurements. This command returns an error when the routine is not in the Ready state.

Response

Key	Type	Description
state	string	

Example Request

```
{ "command": "StartMeasurement" }
```

Example Response

```
{ "state": "OK" }
```

Command: StopMeasurement

Description: Stops the measurements. This command returns an error when the routine is not in the Starting or Running state.

Response

Key	Type	Description
state	string	

Example Request

```
{ "command": "StopMeasurement" }
```

Example Response

```
{ "state":"OK" }
```

Command: CloseRoutine

Description: Stops any ongoing measurement, clears all errors and close the routine.

Response

Key	Type	Description
state	string	

Example Request

```
{"command": "CloseRoutine" }
```

Example Response

```
{ "state":"OK" }
```

Command: GetTestData

Description: Returns the measurement data. The format depends on the active routine.
For more details see

Command: GetTestStatus

Description: Returns the current status of the routine.

Response

Key	Type	Description
routine_status	string	Current state of the routine. One of: Initializing, Ready, Starting, Running, Error.

Example Request

```
{"command": "GetTestStatus" }
```

Example Response

```
{ "routine_status":"Running" }
{ "routine_status":"Error", "error":{"code":1000, "message": "Hardware error"}}
```

Command: ClearErrors

Description: Clears any errors when the routine is in error. After this command, the routine returns to the Ready state.

Response

Key	Type	Description
state	string	

Example Request


```
{"command": "GetTestStatus" }
```

Example Response

```
{ "state":"OK" }
```

Command: GetSettings

Description: Returns the routine configuration. The format depends on the active routine.

Response

Key	Type	Description
state	string	

Example Request

```
{"command": "GetSettings" }
```

Example Response

```
{
  "device_type":"NI-SMU",
  "device_settings":{"Compliance (A)":0.01},
  "scan_settings":{
    "scan":{
      "Start (V):-0.2,
      "End (V)":1,
      "Step (V)":0.02,
      ...
    }
  }
}
```

Command: ApplySettings

Description: Applies a configuration. The configuration JSON must match the active routine. It is recommended to use GetSettings to obtain the correct JSON.

Response

Key	Type	Description
state	string	

Example Request

```
{
  "device_type":"NI-SMU",
  "device_settings":{"Compliance (A)":0.01},
  "scan_settings":{
    "scan":{
      "Start (V):-0.2,
```

```

        "End (V)":1,
        "Step (V)":0.02,
        ...
    }
}

```

Example Response

```
{ "state":"OK" }
```

Command: SetInfo

Description: Set the user name, device name and device area.

Parameters

Key	Type	Required	Description
user_name	string	✓	User name
device_name	string	✓	Device name
device_area	float		Device Area in cm ²

Response

Key	Type	Description
state	string	OK

Example Request

```

{
  "command": "SetInfo",
  "params": { "user_name": "User",
              "device_name": "Sample",
              "device_area": 1.23 },
}

```

Example Response

```
{ "state":"OK" }
```

Routine Specific Commands

Some routines have specific commands. They can only be called when that routine is active. Otherwise an “invalid command” is returned.

The following sections detail the specific commands for each routine.

JV Commands

TBA

TMPCTL

These commands can only be called when a temperature controller is available.

TBA

MUX

These commands can only be called when a device multiplexer is available

Command Summary

Command	Description
Connect	Connects the multiplexer
SetState	Set the state of each multiplexer channel
GetStatus	Gets the status of the multiplexer
Disconnect	Disconnects the multiplexer

Commands Details

Command: Connect

Description: Connects the multiplexer

Response

Key	Type	Description
state	string	Connection status

Example Request

```
{"command": "Connect"}
```

Example Response

```
{ "state":"OK" }
{ "state":"Error", "error":{"code":1000, "message": "Multiplexer not found"}}
```

Command: SetState

Description: Set the state of each multiplexer channel.

Parameters

Key	Type	Required	Description
State	bool[]	✓	Boolean array of state for each channel

Response

Key	Type	Description
state	string	OK

Example Request

```
{
  "command": "SetState",
  "params": { "state": [false, true, false]}
}
```

Example Response

```
{ "state": "OK" }
{ "state": "Error", "error": {"code": 1000, "message": "Unable to set channels"}}
```

Command: GetStatus

Description: Gets the status of the multiplexer.

Response

Key	Type	Description
connected	Boolean	Whether the multiplexer is connected
firmware	String	Firmware of the connected multiplexer
nrRelays	Int	Number of available channels
state	Bool[]	Boolean array of state for each channel

Example Request

```
{ "command": "GetStatus" }
```

Example Response

```
{
  "connected": true,
  "firmware": "Multiplexer_v1.1",
  "nrRelays": 4,
  "status": [false, false, false, true]
}
```

Command: Disconnect

Description: Disconnects the multiplexer.

Response

Key	Type	Description
state	string	OK

Example Request

```
{ "command": "Disconnect" }
```

Example Response

```
{ "state": "OK" }
```

Examples

Python

```
import socket
import json

def exampleCommand(command, data=''):
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as sock:
        sock.connect(('localhost', 6340))

        message = json.dumps({'command': command, 'data': data})

        # Prepare the length of the string
        length = len(message)
        length_bytes = length.to_bytes(4, byteorder='big') # 4 bytes to
represent the length

        sock.sendall(length_bytes) # Send the length of the string
        sock.sendall(message.encode('utf-8')) # Send the string

        # Receive response
        response = sock.recv(1024)
        print('Received:', response.decode('utf-8'))

exampleCommand('GetAvailableRoutines')
```


Changelog

v0.1

Initial version